

NetComms 2 Summative 2 Reviewer: DevNet Modules 3 and 4

DevNet Associate. Combined study notes for Module 3 (Software Development and Design) and Module 4 (Understanding and Using APIs). The blue boxes are memory aids I made; they are not from the slides.

What Is Inside

- **Part 1, Module 3:** SDLC, methodologies, design patterns, Git, coding basics, testing, and XML, JSON, YAML.
- **Part 2, Module 4:** APIs, REST, HTTP methods and status codes, authentication, rate limits, webhooks, troubleshooting.
- Each part ends with a one-page cram sheet. Read the cram sheets last, right before the exam.

Fast Memory Map

Topic	Hook
SDLC phases	Real Dogs Invite Tiny Dragons, Mostly
Lean principles	Even A Dull Day Earns Bright Optimism
Git flow	Edit, add, commit, push (pull is the reverse)
Git states	Modified, Staged, Committed
TDD	Test, Run, Write, Run, Refactor
Data formats	JSON has no comments; YAML is a superset of JSON
REST constraints	Clients Stay Calm Under Layered Code
Request parts	URI, Method, Header, Body
CRUD methods	Create POST, Read GET, Update PUT, Delete DELETE
401 vs 403	401 who are you; 403 I know you, no
Rate limit headers	Limit, Remaining, Reset
Webhook	Reverse API: the app runs, registers, handles

Part 1: Module 3, Software Development and Design

How To Use These Notes

Module 3 has six topics: **3.1 Software Development, 3.2 Design Patterns, 3.3 Version Control (Git), 3.4 Coding Basics, 3.5 Code Review and Testing, 3.6 Data Formats**. Git commands and the three data formats are the usual exam targets, so spend the most time on those.

3.1 Software Development

The Software Development Life Cycle (SDLC)

SDLC is the process of building software **from an idea to delivery**. It is more than coding: it also includes gathering requirements, making a proof of concept, testing, and fixing bugs. It has **six phases**, and **each phase takes input from the result of the previous one**.

#	Phase	What Happens	Output
1	Requirements and Analysis	Explore stakeholders' situation, needs, constraints, and infrastructure. Analyze: is it possible and on budget? Risks to the schedule? How will it be tested? When and how is it delivered?	SRS (Software Requirement Specification) in classic waterfall, confirmed with stakeholders
2	Design	Architects and developers design the software from the SRS	HLD (High-Level Design) and LLD (Low-Level Design) documents
3	Implementation	Also called the coding or development phase. All components and modules are built. The longest phase.	Functional code implementing all requirements, ready to test
4	Testing	Code is installed in a test environment. Functional, integration, performance, and security testing. Continues until the code is bug free.	High-quality working software ready for production
5	Deployment	Software is installed into the production environment	Product manager releases the final software to end users
6	Maintenance	Support customers, fix bugs found in production, improve the software, gather new requests	Team starts the next iteration and version

Memory Aid

Sentence: **Real Dogs Invite Tiny Dragons, Mostly.**

= **R**equirements, **D**esign, **I**mplementation, **T**esting, **D**eployment, **M**aintenance.

Output pairs: Requirements gives **SRS**. Design gives **HLD + LLD**. Implementation is the

longest.

SRS = what to build. HLD = big picture. LLD = detail.

Three Popular Methodologies

A software development methodology is also called an **SDLC model**. The three most popular are **Waterfall, Agile, and Lean**. Which one to use depends on the **type of project, length of project, and size of the team**.

Model	Key Facts From The Slides
Waterfall	Original model created by Winston W. Royce ; the original had seven phases (shown as a diagram on the slide). Still widely used but gradually being replaced by more adaptive methods.
Agile	Flexible and customer-focused. A group of 17 developers wrote the Agile Manifesto in 2001 .
Lean	Based on Lean Manufacturing : minimize waste, maximize value to the customer.

Agile

The Four Values

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

Memory Aid

Left side only: **People, Product, Partners, Pivot.**

The Twelve Principles (Cisco Keywords)

Customer focus; Collaboration; Working software; Simplicity; Embrace change and adapt; Motivated teams; Work at a sustainable pace; Self-organizing teams; Frequent delivery of working software; Face-to-face conversations; Agile environment; Continuous improvement.

Popular Agile Methods

Method	Focus
Agile Scrum	Small, self-organizing teams that meet daily for short periods and work in iterative sprints
Lean	Eliminate wasted effort in planning and execution; reduce programmer cognitive load
Extreme Programming (XP)	Addresses the quality-of-life issues of development teams
Feature-Driven Development (FDD)	Work from an overall model, then break out, plan, design, and build feature by feature

Memory Aid

Four Agile methods: **S-L-X-F** = Scrum, Lean, XP, FDD. "Some **L**ions **X**-ray **F**ish."

Sprints, Backlog, And User Stories

- **Sprint:** a fixed period, usually **2 to 4 weeks**. The team takes as many tasks (user stories) as it can finish. At the end the software should be **working and deliverable**. Length is set before starting and rarely changes.
- **Backlog:** all the features of the software in a **prioritized list**.
- **User story:** a simple statement of what a user (or role) needs, and why. Small enough for one team to finish in one sprint.
- **Scrum team:** cross-functional, collaborative, self-managed, **no more than 10 people**. The scrum master holds a **daily stand-up**, same time every day, **no more than 15 minutes**: what is finished, in progress, or about to start.

User Story Template

As a <user or role>, **I would like to** <action>, **so that** <value or benefit>.

Memory Aid

Template = **Who, What, Why**. As a WHO, I want WHAT, so that WHY.

Lean: Seven Principles (Cisco Wording)

Principle	Meaning
Eliminate waste	The most fundamental lean principle
Amplify learning	Short sprints of working software: developers learn faster, customers give feedback sooner, features are adjusted to bring more value
Decide as late as possible	Under uncertainty, delay decisions so they rest on facts, not opinions or speculation
Deliver as fast as possible	Enables feedback, amplifies learning, makes decisions faster, gives required features, produces less waste. It does not allow customers to change their mind.
Empower the team	Each person decides in their own area of expertise
Build integrity in	The software addresses the customer's needs and stays useful to them
Optimize the whole	Build cohesively; if each expert looks only at their own part, the value of the whole suffers

Memory Aid

Even **A Dull Day Earns Bright Optimism** = Eliminate, Amplify, Decide late, Deliver fast, Empower, Build integrity in, Optimize the whole.

The Seven Wastes

- Partially done work
- Extra processes
- Extra features
- Task switching
- Waiting
- Motion
- Defects

Memory Aid

Please Eat Every Tasty Waffle, My Darling = Partial work, Extra processes, Extra features, Task switching, Waiting, Motion, Defects.

3.2 Software Design Patterns

Design patterns are **best-practice solutions to common problems** and are **language independent**. In 1994, **Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides** (the **Gang of Four, GoF**) published **Design Patterns: Elements of Reusable Object-Oriented Software**. Two key ideas from it: **program to an interface, not an implementation**, and **favor object composition over class inheritance**.

The GoF listed **23 patterns** in **three categories: Creational, Structural, Behavioral**. The two most common patterns named in the module are the **Observer** (a Behavioral pattern) and **MVC**.

Memory Aid

GoF categories: **Create, Structure, Behave** (Creational, Structural, Behavioral).
Gang of Four = **4 names, 23 patterns, 1994**.

Observer Pattern

A **subscription and notification** design: objects receive events when something they are observing changes. To work, the **subject** must **store a list of its observers** and have methods to **add and remove** observers. Benefit: observers get **real-time data** when a change happens. Subscription mechanisms perform better than alternatives such as **polling**.

Memory Aid

Observer = **YouTube subscribe + bell**. You do not keep checking (polling). The channel (subject) keeps a list of subscribers and notifies them.

Model-View-Controller (MVC)

MVC simplifies apps that depend on **graphical user interfaces** by splitting code and responsibility into three parts. Benefit: each component can be **built in parallel**.

Part	Job
Model	The application's data structure . Manages the data, logic, and rules . Gets input from the controller.
View	The visual representation of the data
Controller	The middleman between model and view. Takes in user input and manipulates it to fit the format for the model or view.

Memory Aid

M = Memory (data and rules). **V = Visible** (what you see). **C = Connector** (carries user input between them).

Watch Out

The slide says the controller reshapes input "to fit the format for the model or view". On Dawn's quiz key the accepted answer for "the controller manipulates user input to fit the format for ____" was **the model**. Read the answer choices: if both "model" and "model or

view" appear, follow the quiz key (model).

3.3 Version Control Systems

Version control (also revision control or source control) manages changes to a set of files and keeps a history.

Five benefits: enables **collaboration, accountability and visibility, work in isolation, safety, work anywhere.**

Three Types

Type	Model	Key Facts
Local (LVCS)	A simple local database	Stores the delta between versions; reverse the delta to get an older version
Centralized (CVCS)	Server-client ; repository on a central server	Only one person edits a file at a time. Check out to lock , check in when done
Distributed (DVCS)	Peer-to-peer ; repository usually on a hosting service	Everyone works on any file at the same time, no locking . Push to the main repo; the system detects conflicts

Memory Aid

Local = on my laptop only. Centralized = one Chair, one person at a time (lock). Distributed = Dance floor, everyone at once (no lock).

Git

Git is an **open source DVCS**. Key difference from other systems: Git stores **snapshots, not differences (deltas)**. If a file did not change, Git stores a **reference link** to the last snapshot instead of a new copy.

Three Stages And Three States

Three Stages (Places)	Three States
Repository (the .git directory)	Committed
Working directory	Modified
Staging area	Staged

Memory Aid

A file travels: **Modified** (in the working directory) then **Staged** (in the staging area) then **Committed** (in the repository).

Words: **Work, Stage, Store**. Edit it, stage it, store it.

Local Vs Remote Repositories

- **Local:** on your own machine, where you run the git commands.
- **Remote:** somewhere else, usually a server or hosting service. It still counts as DVCS because it holds the **full repository (code and history)**.
- Cloning gets the full repository **without locking**.
- After cloning (or creating the remote from local), the two are **independent** until you apply changes with a **manual Git command**.

Git Vs GitHub

Git is the distributed version control implementation with a command-line interface. **GitHub** is a **service provided by Microsoft** that hosts repositories with Git and adds code review, documentation, project management, bug tracking, and feature requests. GitHub introduced the **pull request**: a formal request to review a contributor's branch changes for inclusion in the main branch.

Git Commands

Command	What It Does
git config --global key value	Sets the initial global settings
git init <project directory>	Creates an empty repository or makes an existing folder one. Creates a hidden .git folder (metadata, compressed files, commit history, staging area) and the master branch.
git clone <repository> [target directory]	Gets an existing repository. Four transport protocols: Local, SSH, Git, HTTP .
git status	Lists files that differ between the working directory and the parent branch
git diff	Generic file comparison; the file does not need to be tracked by Git
git add <file>	Adds files to the staging area . git add . adds all changed files. Only the files you name are added.
git rm <file>	Removes the file from the working directory and stages the removal. Does not work if the file is already staged with changes.
git rm --cached <file>	Removes from the staging area without deleting the file from the working directory
git commit -m "<message>"	Combines the staged changes into one commit and updates the local repository
git push origin <branch>	Sends local commits to a branch of the remote. Fails if there is a conflict . git push origin master for master.
git pull or git pull origin <branch>	Manually updates your local copy from the remote (local copies do not update automatically)
git branch	Lists local branches (also git branch --list). Also creates branches.
git checkout -b <branch>	Creates and switches to a branch

git branch -d <branch>	Deletes a branch
git merge <branch>	Merges a branch into the current branch. Several branches can be listed.

Memory Aid

Edit, Add, Commit, Push. Local work moves **add** (to stage) then **commit** (to local repo) then **push** (to remote). **Pull** is the reverse direction.

Push = up, Pull = down.

-d deletes a branch; **-b** in checkout means **build** a new branch and switch.

What git pull Does (Four Steps)

1. The local repository (.git) is updated with the latest commits and history from the remote.
2. The working directory and branch are updated with that content.
3. A single commit is created on the local branch with those changes.
4. The working directory is updated with the latest content.

Branching And Merging

- Branching lets you work **independently** without affecting the main code. A new repository starts on a branch called **Master**.
- Branches can be local or remote, deleted, and each has its **own history, staging area, and working directory**. Creating and switching is **lightweight and almost instant**.
- Switching branches changes the working directory and staging area but **not** the .git repository.
- In a merge, **only the target branch changes**. The **source branch stays the same**.
- **Fast-forward merge:** Git applies the source commits to the target **automatically, without conflicts**.
- **Merge conflict:** Git cannot fast-forward because it does not know how to combine the changes to the file(s).

Reading A .diff File

Symbol	Meaning
+	The line was added
-	The line was removed
/dev/null	A file was added or removed
blank (space)	Context lines around the changes
@@	Marks that the next block of changes is starting (a file can have several)
index	Shows the commits compared

Memory Aid

Plus = Put in, Minus = Missing now.

3.4 Coding Basics

Clean Code

Clean code is written so **other developers can read and understand it**. It follows common principles on formatting, organization, intuitive components, purpose, and reusability, and stresses **standardization, proper organization, modularity, and inline comments** so it is self-documenting.

Why write it: easier to understand, more compact and organized; modular code is easier to **unit test**; standardized code is easier to **scan with automated tools**; and it looks nicer.

Functions, Methods, Modules, Classes

Term	Meaning
Function	A standalone block of code that performs a task when executed. Write once, run many times. Code that does a discrete task, or is used more than once, is a candidate to encapsulate.
Method	A block of code associated with an object , typically in object-oriented programming
Parameters and arguments	Add flexibility to functions. Parameters are the placeholders in the definition; arguments are the values you pass in.
return	Ends the function and sends a value back to the caller. Any code below the return is skipped.
Module	A set of functions packaged as a single file , expected to work independently, with an interface for other modules. Used to divide a large project (example: circleClass.py).
Class	Bundles data and functionality . Each class declaration defines a new object type . Can be instantiated many times, each with its own attribute values, and can inherit from existing classes.

Memory Aid

Function = free agent. Method = belongs to an object. "Methods are attached to a class."

Watch Out

Python has **no true private** variables or methods. By convention, a name with a **single leading underscore** (`_`) is treated as private.

3.5 Code Review And Testing

Code Review

Developers (reviewers) look over the codebase, part of it, or specific changes and give feedback. It happens **only after the code changes are complete and tested**. Goal: the final code is easy to

read and understand, follows best practices, is correctly formatted, is free of bugs, has proper comments and documentation, and is clean.

Type	How It Works
Formal code review	A series of meetings to review the whole codebase
Change-based code review	Also called tool-assisted . Reviews code changed because of a bug, user story, feature, commit, etc.
Over-the-shoulder code review	A reviewer looks over the shoulder of the developer who wrote it and gives feedback
Email pass-around	Happens after automatic emails from the source control system when a check-in is made

Memory Aid

F-C-O-E: Formal (meetings, whole code), Change-based (tool-assisted), Over-the-shoulder (live, beside you), Email (auto email on check-in).

Testing

- **Functional testing:** does the software work correctly and behave as intended? From **unit testing** (lowest level) to **integration testing** (higher level).
- **Non-functional testing:** usability, performance, security, resiliency, compliance, localization. Is the software fit for purpose?
- **Unit testing:** detailed functional tests of small pieces (lines, blocks, functions, classes) **in isolation**.
- **Integration testing:** makes sure the individual units **fit together** into a complete application.

Python Test Framework	How It Works
PyTest	Automatically runs scripts that start with test_ or end with _test.py , and within them runs functions beginning with test_
unittest	Different syntax: you subclass the built-in TestCase class and add methods whose names begin with test_

Test-Driven Development (TDD)

Developers capture design requirements as **tests first**, then write software to pass them. The pattern is a **five-step repeating loop**:

1. Create a new test.
2. Run the tests to see if any fail for unexpected reasons.
3. Write application code to pass the new test.
4. Run the tests to see if any fail.
5. Refactor and improve the application code.

Memory Aid

T-R-W-R-R: Test, Run, Write, Run, Refactor. Test comes **before** code. Refactor is always last.

3.6 Understanding Data Formats

REST APIs exchange information with remote services. The **three most popular formats** are **XML, JSON, and YAML**. A common REST pattern has four steps:

1. **Authenticate**, usually by POSTing a user and password and getting back an **expiring token**.
2. **GET** a resource from an endpoint, asking for XML, JSON, or YAML.
3. **Modify** the returned data.
4. **POST (or PUT)** to the same endpoint to change the resource's state, and read the reply to see if it worked.

Compare The Three

Point	XML	JSON	YAML
Stands for	Extensible Markup Language	JavaScript Object Notation	YAML Ain't Markup Language
Family	Derived from SGML ; parent of HTML	Derived from how JavaScript writes object literals	A superset of JSON
File ending	.xml	.json	.yaml or .yml
Structure	Symmetrical tags , names are user-defined	Key/value pairs in braces; arrays in brackets	Indentation ; list items start with a dash and space
Comments	Yes (same as HTML)	No standard comments	Yes , starts with # and a space
Whitespace		Not significant	Significant (shows hierarchy)
Types		Numbers, strings, Booleans, nulls	Numbers, strings, Booleans, nulls

Memory Aid

JSON has no comments. XML and YAML do.

YAML can parse JSON, but JSON cannot parse YAML (**superset**).

YAML files open with --- and end with

XML Extras

- **Prologue**: the first line of the file.
- **Attributes**: extra information placed inside a tag.
- **Namespaces**: identified by URIs, set with the **xmlns** attribute (example on the slide: NETCONF 1.0).
- Special characters are encoded because data is readable text.

Parsing Vs Serializing

Term	Meaning
Parsing	Analyze a message, break it into parts, and understand their purpose.

	Turns XML, JSON, or YAML text into a data structure.
Serializing	Roughly the opposite: turns an internal data structure into a formatted character string for sending.

Memory Aid

Parse = Pull apart. Serialize = Send out.

One-Page Cram Sheet

Topic	Remember
SDLC phases	Requirements, Design, Implementation (longest), Testing, Deployment, Maintenance
SRS / HLD / LLD	SRS after requirements; HLD and LLD after design
Methodologies	Waterfall (Royce), Agile (17 developers, 2001), Lean
Agile methods	Scrum, Lean, XP, FDD
Sprint / team / stand-up	2 to 4 weeks / max 10 people / max 15 minutes
User story	As a ___, I would like to ___, so that ___
GoF	4 authors, 1994, 23 patterns, Creational/Structural/Behavioral
Observer	Subject keeps observer list; add and remove; better than polling
MVC	Model = data and rules; View = visual; Controller = middleman
VCS types	Local (delta), Centralized (lock), Distributed (no lock)
Git	Snapshots not deltas; Working directory, Staging area, Repository
Git states	Modified, Staged, Committed
Git flow	add, commit, push; pull to update
Merge	Only target changes; conflict means no fast-forward
Code review types	Formal, Change-based, Over-the-shoulder, Email pass-around
TDD	Test, Run, Write, Run, Refactor
Data formats	XML (tags), JSON (no comments), YAML (superset of JSON, indentation)

Part 2: Module 4, Understanding and Using APIs

How To Use These Notes

Module 4 has eight topics: **4.1 Introducing APIs**, **4.2 Design Styles**, **4.3 Architectural Styles**, **4.4 REST APIs**, **4.5 Authentication**, **4.6 Rate Limits**, **4.7 Webhooks**, **4.8 Troubleshooting**. The biggest exam targets are the **HTTP methods**, **status codes**, **REST constraints**, and **authentication types**.

4.1 Introducing APIs

An **Application Programming Interface (API)** lets one piece of software **talk to another**. It uses common web-based interactions or protocols plus its own proprietary standards, and it decides **what data, services, and functionality** an application exposes to third parties, so an app can expose things **securely**.

Three uses: **automation** (scripts that do manual tasks), **data integration** (consume or react to another app's data), **functionality** (bring another app's function into your product).

Why popular: APIs are designed into modern products and well tested, and simple languages like **Python** let non-software engineers consume them.

Memory Aid

API uses: **A-D-F** = Automate, Data, Function.

4.2 API Design Styles

	Synchronous	Asynchronous
Response	Replies directly with the data immediately	Replies with no data , just to say the request was received
When	Data for the request is readily available	The request takes time or data is not readily available
Client behavior	The client must wait for the response before running more code	The client keeps running ; the server-side design defines what the client must do later
Benefit	Immediate data, better performance if designed well	Application is not blocked , so better performance
Analogy on slide	Tickets sold first-come, first-served	Request accepted now, result later

Memory Aid

Sync = **Stay and wait** (you wait for the answer). **Async** = **Away, call me back**.

A product can have both, and each API's design is independent of the others.

4.3 API Architectural Styles

Three most popular: **RPC, SOAP, REST**.

RPC (Remote Procedure Call)

A **request-response** model that lets an application call a **procedure in another application**; the method runs and the result is returned. It is an API style that works over different transports: **XML-RPC, JSON-RPC, NFS (Network File System), and SOAP**.

SOAP (Simple Object Access Protocol)

An **XML-based messaging protocol** for communication between applications on different platforms or built in different languages. SOAP is:

- **Independent:** any application can talk to any other, on any operating system
- **Extensible:** features such as reliability and security can be added
- **Neutral:** works over any protocol (HTTP, SMTP, TCP, UDP, JMS)

A SOAP message is an XML document with up to four elements:

Element	Contains
Envelope	The root element of the XML document
Header	Application-specific information such as authorization and SOAP attributes
Body	The data to be transported to the recipient
Fault	Error and/or status information

Memory Aid

SOAP is an **Envelope** holding a **Header, Body, and Fault: E-H-B-F**, "Every Hero Battles Fear."

Think of a letter: envelope outside, header is the address label, body is the letter, fault is the "return to sender" note.

REST (Representational State Transfer)

An architectural style authored by **Roy Thomas Fielding**, with **six constraints** that can be applied to any protocol.

Constraint	Meaning
Client-server	Client and server are independent of each other. The client can be built for many platforms, which simplifies the server.
Stateless	Each request must carry all the information the server needs . The server cannot hold session state .
Cache	Responses must say whether they are cacheable or not . If cacheable, the client may reuse the data for later requests.
Uniform interface	Four principles: identification of resources, manipulation of resources through representations, self-descriptive messages,

	hypermedia as the engine of application state.
Layered system	Hierarchical layers; each layer provides services only to the layer above it and uses services from the layer below.
Code-on-demand	The service can return executable code (for example JavaScript). This one is optional because running third-party code is a security risk.

Memory Aid

Sentence: **C**lients **S**tay **C**alm **U**nder **L**ayered **C**ode.

= **C**lient-server, **S**tateless, **C**ache, **U**niform interface, **L**ayered system, **C**ode-on-demand.

The **last one is the optional one** (code-on-demand).

Uniform interface four: **I-M-S-H** = Identify, Manipulate, Self-describe, Hypermedia.

4.4 Introduction To REST APIs

A REST API communicates over **HTTP** and uses the same concepts as HTTP: requests and responses, verbs, status codes, headers and body.

A REST Request Has Four Parts

Memory Aid

U-M-H-B: URI, Method, Header, Body.

1. URI (Uniform Resource Identifier)

Identifies the resource the client wants to manipulate (also called a URL). Its parts:

Part	Meaning
Scheme	Which HTTP protocol: http or https
Authority	Host and port
Path	The location of the resource on the server
Query	Extra details for scope, filtering, or clarifying the request

Memory Aid

S-A-P-Q: Say A Polite Query. In <https://example.com:443/books?author=x>: https = scheme, example.com:443 = authority, /books = path, ?author=x = query.

2. HTTP Method

Method	Action	Description
POST	Create	Create a new object or resource
GET	Read	Retrieve resource details
PUT	Update	Replace or update an existing resource
PATCH	Partial Update	Update some details of an existing resource
DELETE	Delete	Remove a resource

Memory Aid

CRUD order: Create = **POST**, Read = **GET**, Update = **PUT** (PATCH for a part), Delete = **DELETE**.

"**P**ost **G**ets **P**ut **D**own" = POST, GET, PUT, DELETE in CRUD order.

PUT = whole thing replaced. PATCH = patch a small hole.

3. Header

Name-value pairs separated by a colon: name: value. Two types:

Type	Purpose	Example
Request header	Extra information not related to the message content	Authorization: Basic ... (credentials)
Entity header	Describes the content of the body	Content-Type: application/json

		(format of the body)
--	--	----------------------

4. Body

Holds the data for the resource. **POST, PUT, and PATCH** usually include a body; the body is **optional depending on the method**. If a body is sent, its type **must be stated in the Content-Type header**.

A REST Response Has Three Parts

Memory Aid

S-H-B: Status, Header, Body.

HTTP Status Code Categories

Three digits. The **first digit is the category**; the other two are numbered in order.

Range	Category	Meaning
1xx	Informational	Request received, continuing; responses have no body
2xx	Success	Server received and accepted the request
3xx	Redirection	Client must take additional action
4xx	Client Error	Request has bad syntax or invalid input
5xx	Server Error	Server could not fulfill a valid request

Memory Aid

1 Info, 2 Good, 3 Go elsewhere, 4 Your fault, 5 Server's fault.

Common Status Codes

Code	Message	Meaning	Trick
200	OK	Success, usually with a body	Everything fine
201	Created	Resource was created	Think POST
202	Accepted	Accepted for processing, still in process	Async: "got it, working on it"
400	Bad Request	Not processed because of an error in the request (misspelled resource, JSON syntax)	Malformed
401	Unauthorized	No valid authentication credentials	Who are you?
403	Forbidden	Understood, but rejected ; credentials are known but not enough privilege	I know you, but no.
404	Not Found	The resource path was not found	Wrong path
407	Proxy Authentication Required	Like 401, but you must authenticate with the proxy first	Proxy wants ID

409	Conflict	Conflict with the current state of the resource (edit conflict); retry later may work	Two editors
415	Unsupported Media Type	Body format not supported (XML sent to a JSON-only server)	Wrong Content-Type
429	Too Many Requests	Rate limit exceeded	Slow down
500	Internal Server Error	Unexpected server condition	Server broke
501	Not Implemented	Server lacks the functionality	Not built
502	Bad Gateway	Gateway or proxy got an invalid response from upstream	Bad reply upstream
503	Service Unavailable	Overloaded or under maintenance	Busy or down
504	Gateway Timeout	Gateway or proxy got no timely response from upstream	No reply upstream

Watch Out

401 vs 403 is a favorite. **401** = authentication problem (check username, password, API key, token, URI). **403** = authentication is fine but **not authorized**; fix by using credentials with enough privileges.

502 vs 504: 502 = invalid reply; 504 = no reply in time.

Response Headers And Extras

- **Response headers** (extra info not about the content): **Set-Cookie** (server sends cookies) and **Cache-Control** (directives all caches must obey, for example max-age=3600, public).
- **Entity header** in a response: **Content-Type** (format of the body).
- **Pagination**: data is broken into chunks; usually a **query parameter** picks the page.
- **Compression**: for large data that cannot be paginated. Ask with the **Accept-Encoding** request header. Values: **gzip**, **compress**, **deflate**, **br**, **identity**, and the asterisk wildcard.
- **Sequence diagram** for an API has three sequences on the slide: **Create session** (HTTPS with credentials), **Get devices**, **Create device** (a POST).

4.5 Authenticating To A REST API

REST APIs need authentication so random users cannot access, create, update, or delete data. Some read-only APIs with nothing confidential need none.

	Authentication	Authorization
Question	Who are you?	What are you allowed to do?
Meaning	Proves the user's identity	Defines access : has permission to do the action on that resource
Slide analogy	Showing government ID or biometrics at the airport	Showing your ticket at a concert

Memory Aid

AuthN = Name (identity). AuthZ = Zone (what zones you can enter).

Authentication Mechanisms

Type	How It Works
Basic	Sends username:password separated by a colon and encoded in Base64
Bearer	Uses a bearer token , a string generated by an authentication server such as an Identity Service (IdS)
API Key	A unique alphanumeric string generated by the server and assigned to a user. Two types: public and private .

Memory Aid

B-B-A: Basic, Bearer, API key. Basic = password pair. Bearer = token. API key = assigned string.

Base64 is encoding, not encryption.

Authorization: OAuth

- **Open Authorization (OAuth)** combines authentication with authorization and was created to fix insecure authentication mechanisms.
- Two versions, **OAuth 1.0** and **OAuth 2.0**. **OAuth 2.0 is not backwards compatible.**
- OAuth 2.0 lets a **third-party app get limited access** to an HTTP service.
- The user gives credentials **directly to the authorization server** (an **Identity Provider, IdP, or Identity Service, IdS**) to obtain an **access token** to share with the app. The process of getting the token is called a **flow**.

4.6 API Rate Limits

A rate limit controls **how many requests** a user or app can make per unit of time. It helps to **avoid server overload, give better service to all users, and protect against Denial-of-Service (DoS) attacks.**

Four Algorithms

Algorithm	How It Works
Leaky Bucket	Requests go into a queue in the order received. They can arrive at any rate , but the server processes the queue at a fixed rate . If the queue is full, the request is rejected .
Token Bucket	Each user gets a set number of tokens per time increment. Each request spends one token . No token means the request is rejected. The client must track its tokens.
Fixed Window Counter	A fixed time window has a counter of allowed requests. When the limit is hit, all further requests in that window are rejected .
Sliding Window Counter	Allows a fixed number of requests in a set duration. For each new request the server counts requests made from the start of the window to now .

Memory Aid

Leaky = **steady drip** out of a bucket (fixed outflow). **Token** = **tickets**, spend one per request. **Fixed window** = **a clock that resets** at set times. **Sliding window** = **a moving window** that counts back from right now.

Knowing And Exceeding The Limit

Header Key	Meaning
X-Rate-Limit-Limit	Maximum requests in the time unit
X-Rate-Limit-Remaining	How many requests you can still make in this window
X-Rate-Limit-Reset	When the window resets

When you exceed the limit the server rejects the request and returns an HTTP error. Most common codes: **429 Too Many Requests** or **403 Forbidden**.

Memory Aid

Limit, Remaining, Reset: **L-R-R**. Max, Left, When-again.

4.7 Webhooks

A **webhook** is an **HTTP callback**, an **HTTP POST to a specified URL**, that notifies your app when an event happens. Apps do not need to **poll**. Webhooks are called **reverse APIs** because the app **subscribes** by registering with the webhook provider. **Multiple apps** can subscribe to one webhook server. Examples: Cisco DNA Center webhooks for network events; Webex Teams notifying you of new messages in a room.

To Consume A Webhook, The App Must:

1. Be **running at all times** to receive HTTP POST requests.
2. **Register a URI** with the webhook provider.

3. Handle the incoming notifications.

Memory Aid

Webhook = the pizza shop calls you when it is ready (you do not keep calling to ask, which is polling).

Run, Register, Handle: **R-R-H**.

4.8 Troubleshooting API Calls

Keep the **API reference guide** and the **authentication information** handy before you troubleshoot.

No Response And No Status Code

Side	Causes And Fixes
Client side	User error (mistyped URI); invalid URI (for example missing scheme); wrong domain name ; connectivity issues (proxy, firewall, VPN); SSL/invalid certificate (only with HTTPS, because of the SSL handshake). In a lab where certificates are not valid you can turn off verification with the verify parameter in the Python requests library. Real fix: find the root cause.
Server side	API server powered off , cabling problems , domain name changed , network down . You get a long silence then a traceback . Use a network capture tool and server logs if you have access. Cannot be fixed from the client; contact the server administrator .

Memory Aid

Client causes, **U-I-W-C-S**: User error, Invalid URI, Wrong domain, Connectivity, SSL certificate.

Steps When You Get A Status Code

1. **Check the return code** (print it during development).
2. **Check the response body** (print it too; the server often says what is wrong, for example "No id field provided").
3. Use a **status code reference** if that does not solve it.

The status code is part of **HTTP/1.1 (RFC 7231)**: the **first digit is the class**; the last two digits do not categorize anything.

Common 4xx Fixes

Code	Fix
400	Fix misspelled resources or JSON syntax problems; send required fields (example: the id)
401	Check username , password , API key , token , request URI ; add the authentication to the call
403	Not an authentication issue; the user lacks privileges. Use an account with enough privileges .
407	Authenticate with the proxy first
409	Conflict, such as two people editing; retrying later may work once the server resolves it
415	Send a format the server supports ; add or correct the Content-Type header such as application/json

Summary: 4xx = client-side errors, 5xx = server-side errors. Server-side problems are fixed by the server administrator.

One-Page Cram Sheet

Topic	Remember
API uses	Automation, data integration, functionality
Sync vs Async	Sync returns data now and client waits; Async returns no data first, client keeps going
Architectural styles	RPC, SOAP, REST
SOAP message	Envelope (root), Header, Body, Fault
REST author	Roy Fielding; six constraints; code-on-demand is optional
REST constraints	Client-server, Stateless, Cache, Uniform interface, Layered, Code-on-demand
Request parts	URI, Method, Header, Body
URI parts	Scheme, Authority (host and port), Path, Query
Methods	POST create, GET read, PUT update, PATCH partial, DELETE delete
Response parts	Status, Header, Body
Codes	200 OK, 201 Created, 202 Accepted, 400, 401, 403, 404, 500, 503; 429 too many requests
401 vs 403	401 who are you; 403 I know you, no
Auth types	Basic (Base64), Bearer (token), API key; OAuth does authorization
Rate limit algorithms	Leaky bucket, Token bucket, Fixed window, Sliding window
Rate limit headers	Limit, Remaining, Reset
Webhook	HTTP POST callback, reverse API, no polling; app must run, register, handle
Troubleshooting	Check code, check body, use reference; 4xx client, 5xx server